

浙江大学



分布式智能物资管理系统

基于 YOLO 识别、身份门控与本地库存看板的嵌入式课程设计

姓 名 黄鹤翔

学 号 3230106231

院 系 信息与工程学院

指导老师 杜歆

2026 年 4 月

摘要

本项目面向实验室物资借还场景，设计并实现了一套分布式智能物资管理系统。系统由客户端（采集与交互）和服务端（识别与记录）组成。客户端为独立 Python 包，基于树莓派和普通 Linux 摄像头采集图像，通过 TCP 协议向服务端发送请求；服务端内部使用 ROS2 节点组织人脸识别、YOLO 物资计数、库存记录和 Web 看板，并通过多线程模型支持多个客户端并发连接。一次完整出入库操作由”人脸门控—前态拍摄—用户操作—后态拍摄—自动差分—记录写入”六步串联。当前模型在 cboard、dmj4310、m3508 三类物资上 mAP50 达到 87.74%，mAP50-95 达到 76.15%，能够支撑系统联调和较为简单的物资管理。

关键词：嵌入式系统；物资管理；YOLO；人脸识别；TCP 协议；ROS2；库存看板；分布式系统

目录

摘要	1
一、背景与需求	4
1.1 问题提出	4
1.2 核心需求	4
二 系统设计	5
2.1 总体架构	5
2.2 主流程设计	6
2.3 技术选型	8
三 关键技术实现	9
3.1 客户端：轻量化与状态机	9
3.2 客户端部署与自启动	11
3.3 TCP 协议与跨设备通信	12
3.4 服务端：分布式多客户端并发	15
3.5 ROS2 节点化识别	18
3.6 物资检测	19
3.7 人脸识别门控	20
3.8 库存记录与看板	21
四 数据集与模型训练	22
4.1 数据准备	22
4.2 训练过程与结果	23
五 项目迭代历程	24
5.1 版本演进	24
5.2 关键取舍	26
六 系统评估	26
6.1 系统特点	26
6.2 不足与改进方向	27

七 总结	27
参考资料	29

一、背景与需求

1.1 问题提出

小型实验室和机器人队伍中共享电机、电调、开发板等物资是常态。纸质表格或电子表格在借还频率低时勉强可用，但随着借用增多，三个问题逐渐突出：

- (1) 物资被取走但忘记登记，记录与实物不符；
- (2) 需要有专人进行物资的管理和定期清点，耗费人力；
- (3) 记录散落在不同文件中，难以汇总查看当前库存。

例如在如下战队的物资管理中，库中记录还有大量的 C620 电调，但是实际上队内已经没有能够使用的 C620 电调，并且该登记表由于无人管理已经多个月没有进行更新

☐	物资编号	物资类型	当前状态	最后出/入库人	最后出/入库时间
2	C620_2329	C620	在库内		
3	C620_2333	C620	已出库	高恒斌	
4	C620_2401	C620	已损坏		
5	C620_2216	C620	在库内		
☐	C620_2601	C620	已出库	陈庄润	
7	C620_2602	C620	在库内		
8	C620_2603	C620	已出库	罗来宇	
9	C620_2604	C620	已出库	高恒斌	
10	C620_2605	C620	在库内		
11	C620_2606	C620	已出库	陈庄润	
12	C620_2607	C620	已出库	高恒斌	
13	C620_2608	C620	已损坏		
14	C620_2609	C620	已出库	高恒斌	
15	C620_2610	C620	在库内	方贝宁	
16	C620_2611	C620	在库内		
17	C620_2612	C620	在库内		
18	C620_2613	C620	在库内		
19	C620_2614	C620	在库内		
20	C620_2615	C620	在库内		

1.2 核心需求

系统需要满足四条核心需求：

- (1) 操作前完成身份确认，方便对物资进行溯源；
- (2) 尽量实现自动化，减少队员出入库物资的操作；

(3) 客户端保持轻量，能在树莓派等轻量化普通 Linux 环境中直接运行；

(4) 服务端记录每次出入库流水，并提供本地看板便于检查。

围绕这些需求，系统按”客户端采集—网络通信—服务端识别—记录展示”四层划分。客户端负责摄像头预览和用户交互，服务端负责推理、记录和展示，两层之间通过 TCP 长连接通信。

二、系统设计

2.1 总体架构

系统架构如图 1 所示。客户端侧包含 CameraNode、Tkinter UI、LogicNode 和 TcpClientNode 四个模块；服务端侧以 TcpBridgeNode 为入口，内部将人脸识别、物资识别、库存记录和 Web 看板拆为独立的 ROS2 节点。

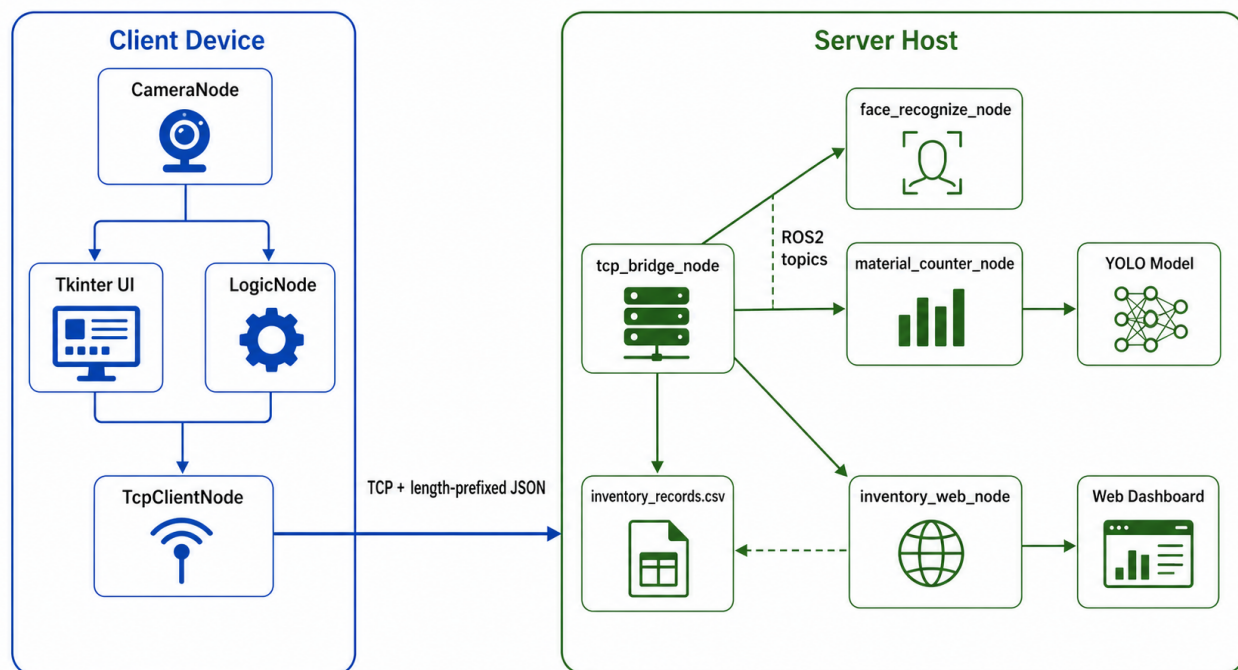


图 1: 系统总体架构

客户端与服务端的边界很明确：客户端只做采集和交互，通过 TCP 发送 face_image、material_image、inventory_record 三类请求；服务端收到图片类请求后转成 ROS2 Image 消息发布到识别节点，识别结果以 JSON 返回；库存请求经校验后直接追加到 CSV 文件。Web 看板节点周期性读取同一份 CSV，构建当前库存、最近事件和低库存提示。

这种结构把压力分布在合适的位置：树莓派只做采集和交互，服务器负责推理和汇总；客户端只需要轻量化的 python 环境，服务端利用 ROS2 对多个模块进行良好地组织。

2.2 主流程设计

一次出入库操作的主流程如图 2 所示，核心设计是”先身份、再前态、再后态、最后差分”。

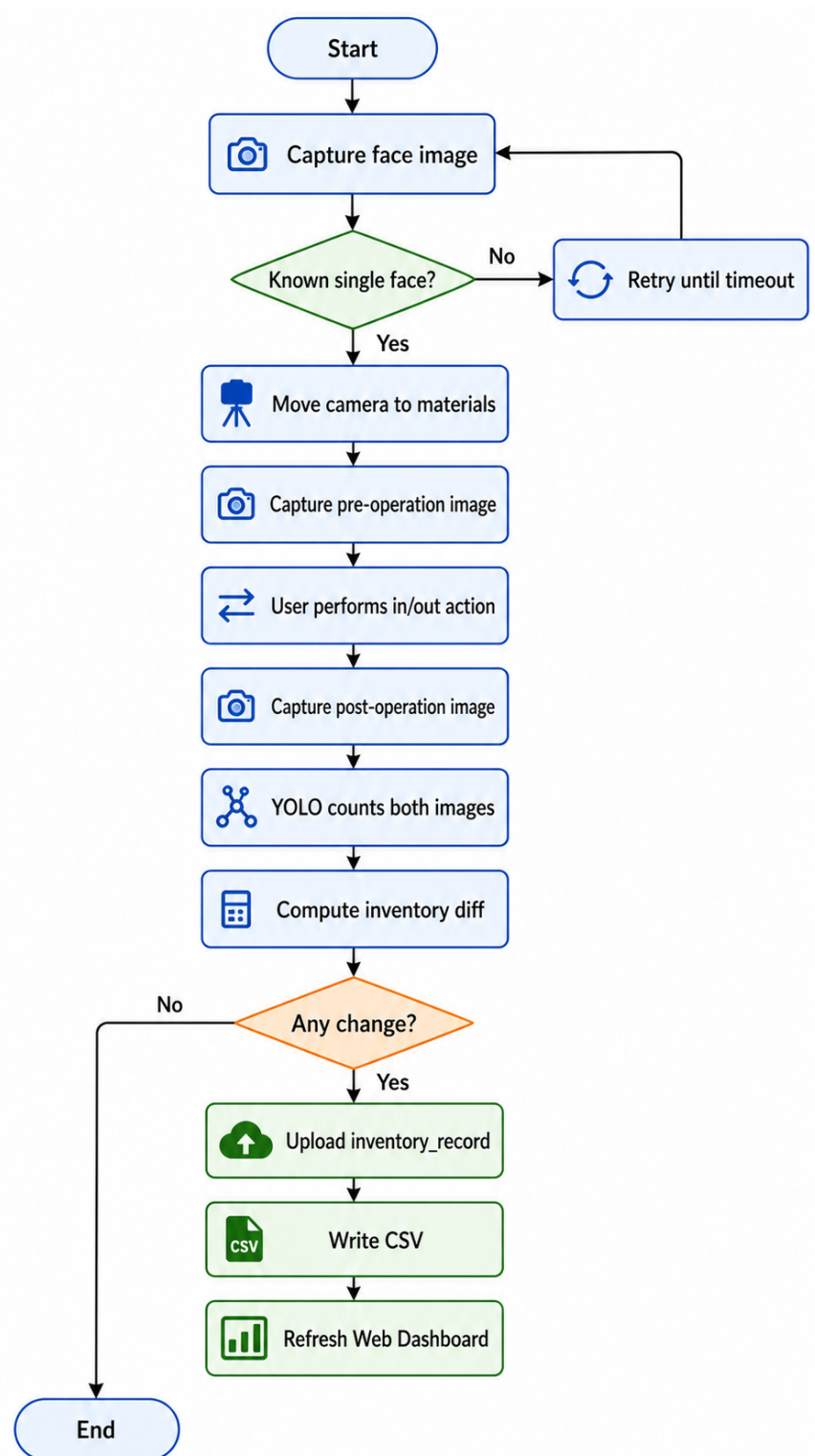


图 2: 出入库主流程

客户端初始状态为 `idle`，一次完整流程可概括为以下几步：

- (1) 用户点击 Start, 客户端进入 recognizing_face, 每隔约 1 秒抓拍一张人脸图发往服务端;
- (2) 只有当服务端返回 total_faces=1 且唯一人脸不是 unknown 时, 才记录操作者姓名并进入 waiting_camera_move;
- (3) 用户将摄像头对准物资区域, 点击 Capture Materials, 拍摄前态图像;
- (4) 用户完成实际借还操作后点击 Finish, 客户端拍摄后态图像;
- (5) 两次 material_image 返回后, compute_inventory_changes() 对前后计数字典逐项求差:
 - 正差生成入库条目;
 - 负差生成出库条目;
 - 若差分为空, 则流程直接结束, 不写入无意义流水。

前态拍摄采用手动触发而非自动判断, 是因为早期尝试通过画面稳定性自动触发, 但当前系统只有一个摄像头, 在移动摄像头和现场遮挡场景下误拍率较高, 改为手动确认后流程更可控。

2.3 技术选型

表 1 概括了各关键技术点的选型理由。

表 1: 关键技术选型

技术点	基本原理	选型理由
ROS2 节点	将功能拆为独立节点，通过 topic 交换消息	服务端推理任务集中，适合用 ROS2 管理人脸、物资、网桥和看板；模块可单独启动和替换。旧版 <code>integrated_server.py</code> 725 行，当前最大服务端节点 357 行。
TCP 长连接	可靠有序字节流，需自定义消息边界	客户端不依赖 ROS2 和 DDS 发现，使用 TCP 进行无线通信，确保客户端在空间上部署地简易性
YOLO 单阶段检测	一次前向推理直接预测目标框、类别和置信度	相比逐件扫描二维码，YOLO 只需要拍一张物资总体图即可统计类别数量。
人脸识别门控	编码比对 + 距离阈值	库存记录必须绑定责任人。要求恰好一张已知人脸才放行，过滤无人、多人和未知人员场景。
有限状态机	将流程拆为有限状态，每个状态只允许特定转移	出入库动作为顺序性操作，状态机让用户不能跳步，异常重试和超时处理更明确。
CSV + 本地看板	追加式文本记录；Web 看板周期性读取并聚合	课程阶段并发低、调试频繁，CSV 便于直接检查；看板补足展示能力，从同一份 CSV 聚合数据。

三、关键技术实现

3.1 客户端：轻量化与状态机

- (1) **仿 ROS2 的分层结构**。客户端最终部署在树莓派上，因此没有继续引入 ROS2 运行时，而是保留了类似 ROS2 的模块拆分方式：相机采集、业务流程、TCP 通信和图形界面分别放在 `camera_node`、`logic_node`、`tcp_client_node` 和 `ui_node` 中。这样做的好处是，一方面客户端安装时只需要普通 Python 环境；另一方面服务端同学阅读代码时，仍然能快速对应到“节点职责”这一层次。
- (2) **相机采集原理**。摄像头通过连续抓取视频帧为 UI 预览提供流式画面，同时支持在业务请求时捕获当前帧作为识别输入。由于客户端部署在树莓派上，且相机使用排针相机，因此相机后端采用自动选择策略：若环境中存在 `rpikam-still`，通过命令行输出 JPEG 到标准输出并在内存中解码；否则回退到 OpenCV。选择 `rpikam-still` 的原因是树莓派新系统

中它比直接打开 OpenCV 摄像头更稳定，回退路径保证普通 USB 摄像头也能用于开发调试。

- (3) **有限状态机原理**。有限状态机 (Finite State Machine, FSM) 将系统行为拆为有限个状态，每个状态只允许特定的输入和状态转移。在本系统中，出入库操作天然具有顺序性——必须先完成身份认证才能拍摄物资、必须先拍前态才能拍后态——FSM 正好保证用户不能跳步执行。
- (4) **具体实现**。流程控制基于 FSM, LogicNode 在 `client/client/logic_node.py` 中实现。核心状态包括：

- `recognizing_face`;
- `waiting_camera_move`;
- `capturing_pre_material`;
- `waiting_finish`;
- `capturing_post_material`;
- `computing_diff`;
- `submitting_inventory`。

每个状态只开放当前合理的操作：

- `recognizing_face` 阶段只能等待身份确认；
- `waiting_camera_move` 阶段只允许用户手动拍摄前态物资；
- `waiting_finish` 阶段才允许点击 Finish。

UI 层不直接处理识别细节，只负责把三个按钮事件交给 LogicNode：

Listing 1: 客户端状态机核心入口

```
1 def start_operation():
2     reset workflow data
3     state = "recognizing_face"
4
5 def capture_material_now():
6     require state == "waiting_camera_move"
7     capture material snapshot as "pre"
8
9 def finish_operation():
10    require state == "waiting_finish"
11    record finish time
```

12 capture material snapshot as *"post"*

- (5) **库存差分原理**。差分计算的核心思想是：操作前计为基准 P ，操作后计为 Q ，对于每个物资类别 i ，变化量 $\Delta_i = Q_i - P_i$ 。 $\Delta_i > 0$ 时入库 $|\Delta_i|$ 件， $\Delta_i < 0$ 时出库 $|\Delta_i|$ 件， $\Delta_i = 0$ 时不产生记录。该逻辑在 `client/client/logic_utils.py` 中实现：

Listing 2: 库存差分逻辑

```
1 for name in sorted(set(pre_counts) | set(post_counts)):
2     pre_value = int(pre_counts.get(name, 0))
3     post_value = int(post_counts.get(name, 0))
4     delta = post_value - pre_value
5     if delta > 0:
6         added_items.append({"name": name, "quantity": delta})
7     elif delta < 0:
8         removed_items.append({"name": name, "quantity": -delta})
```

3.2 客户端部署与自启动

客户端除了“能运行”，还要解决“每次上电后如何尽快进入可用状态”的问题。课程演示时如果树莓派每次重启都需要重新激活环境、手动输入启动命令，现场使用体验会很差，因此本项目把自启动也纳入了端侧实现的一部分。

- (1) **解释器自动选择**。自启动脚本位于 `client/scripts/start_client_autostart.sh`。考虑到开发阶段可能同时存在 `conda`、项目本地虚拟环境和系统 `Python`，脚本按以下顺序寻找解释器：

- (1) 环境变量 `CLIENT_PYTHON_BIN`;
- (2) `client/.venv/bin/python`;
- (3) `/miniconda3/envs/alg/bin/python`;
- (4) `python3`。

这样部署机器即使环境略有差异，也不需要反复改脚本主体。

- (2) **启动前自检与日志**。脚本在真正拉起客户端前，会把启动时间、实际使用的 `Python` 路径、解释器版本以及 `cv2/tkinter` 导入结果追加到 `client_autostart.log`。这一设计看似简单，但在排查“手动能跑、开机自启却失败”这类问题时非常有用，因为多数故障并不来自业务逻辑，而是来自自启动时用错了解释器或缺少图形环境依赖。

- (3) **桌面自启动而不是后台服务**。由于客户端界面使用 `tkinter`，它依赖图形桌面会话，不适合直接做成普通后台 `systemd` 服务。因此本项目采用树莓派桌面环境的 `/.config/autostart/*.desktop` 方式拉起图形客户端，再配合 Raspberry Pi OS 的桌面自动登录，使设备上电后能自动进入桌面并显示客户端窗口。相比后台守护进程，这种方案更贴合课程项目的实际使用场景，原因主要有两点：
- 操作者需要立刻看到预览画面；
 - 操作者需要直接看到状态提示和三个控制按钮。
- (4) **实际收益**。自启动机制补齐了系统部署链路中的最后一环。对于客户端而言，用户的操作被压缩为“上电、等待桌面进入、开始使用”，不需要了解 ROS2、Python 环境或命令行参数；对于维护者而言，服务端地址、端口和相机后端仍然保留在脚本入口处，后续更换网络环境时修改也比较集中。

3.3 TCP 协议与跨设备通信

- (1) **协议设计原理**。TCP 是面向连接的可靠字节流协议，保证数据按序、无差错到达，但 TCP 本身不提供消息边界，“一次 `send`”不一定对应“一次 `recv`”。因此应用层必须自定义分帧机制。常见方案有三种：
- (1) 固定长度消息；
 - (2) 分隔符方案，如换行符；
 - (3) 长度前缀。

本系统选择方案 (3)，原因也比较直接：

- 图片数据可能包含任意字节序列，分隔符不可靠；
 - 固定长度不适合大小不一的图像请求。
- (2) **双长度前缀格式**。每条消息格式为“4 字节 JSON 长度（大端）+ 4 字节附件长度（大端）+ UTF-8 JSON 头 + 可选二进制图像体”。收端先读 8 字节固定头得到两段长度，再按长度分别读取 JSON 和附件。JSON 头无换行符、保持紧凑，图像体作为独立二进制附件发送，不再将图片 `base64` 嵌入 JSON。
- (3) **帧头编解码实现**。双长度前缀由 `struct.Struct("!II")` 打包和解包，`!` 表示大端字节序，`II` 表示两个无符号 32 位整数。收端的 `recv_exactly()` 循环读取直到收满指定字节数，以此防御 TCP 分包：

Listing 3: TCP 消息收发（两端共用的 `tcp_protocol.py`）

```

1 MESSAGE_PREFIX = struct.Struct("!II")
2
3 def recv_exactly(sock, size):
4     chunks = bytearray()
5     while len(chunks) < size:
6         data = sock.recv(size - len(chunks))
7         if not data:
8             raise ConnectionClosedError("Peer closed the connection")
9         chunks.extend(data)
10    return bytes(chunks)
11
12 def receive_message(sock):
13     header = recv_exactly(sock, MESSAGE_PREFIX.size)
14     json_len, attachment_len = MESSAGE_PREFIX.unpack(header)
15     body = recv_exactly(sock, json_len)
16     attachment = recv_exactly(sock, attachment_len) if attachment_len
17                    else b""
18     payload = json.loads(body.decode("utf-8"))
19     return payload, attachment
20
21 def send_message(sock, payload, attachment=b''):
22     encoded = json.dumps(payload, ensure_ascii=False).encode("utf-8")
23     sock.sendall(MESSAGE_PREFIX.pack(len(encoded), len(attachment))
24                  + encoded + attachment)

```

(4) **请求分发**。得益于校园网的存在，server 端和 client 端只需要处于同一个校网环境下即可进行 TCP 通信，不需要放在同一个地点。服务端支持三类请求，TcpBridgeNode 根据 type 字段分发：

- material_image：发送到物资识别节点；
- face_image：发送到人脸识别节点；
- inventory_record：直接校验并写入库存流水。

Listing 4: TCP 网桥分发逻辑

```

1 if request_type == "material_image":
2     image = decode_image_payload(payload, attachment)
3     return submit_image_request(material_publisher, request_id, image)
4

```

```

5 if request_type == "face_image":
6     image = decode_image_payload(payload, attachment)
7     return submit_image_request(face_publisher, request_id, image)
8
9 if request_type == "inventory_record":
10     record = validate_inventory_payload(payload)
11     data = write_inventory_record(request_id, record)
12     return make_success_response("inventory_record_result", request_id,
        data)

```

(5) **传输优化与实测**。当前客户端默认将图片编码为 webp，并以二进制附件直接挂在 TCP 报文后面；服务端在 `decode_image_payload()` 中同时兼容旧版 base64 + jpeg 结构，便于平滑升级。为了验证改动是否确实减少了网络负担，本文选取仓库中的 13 张样本图（11 张物资图、2 张人脸图），按两种协议格式重新打包并统计完整 TCP 报文大小。两种方案分别为：

- 旧方案：JPEG 质量 90，并写入 JSON 的 `image_base64` 字段；
- 新方案：当前默认参数，即 WebP 质量 75 + 二进制附件。

结果如图 3 和表 2 所示：总体平均报文由 2.12 MB 降至 0.28 MB，降幅 87.01%。这个优化的核心不只是“换成了 WebP”，更重要的是：

- 避免了 Base64 带来的额外膨胀；
- 把 JSON 头和图像体分开处理，使服务端解析路径更清晰。

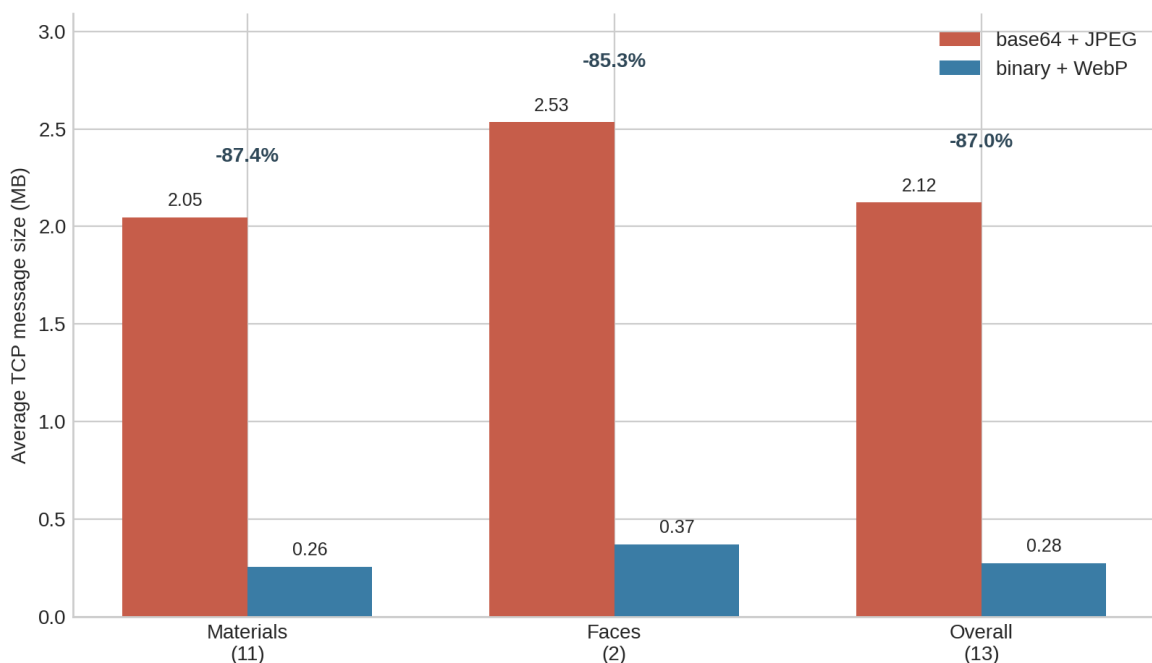


图 3: 旧协议与新协议的平均 TCP 报文大小对比

表 2: 传输优化实测结果

样本集合	旧协议均值 / MB	新协议均值 / MB	降幅
物资图 (11 张)	2.05	0.26	87.40%
人脸图 (2 张)	2.53	0.37	85.28%
总体 (13 张)	2.12	0.28	87.01%

3.4 服务端：分布式多客户端并发

(1) **并发模型原理**。服务端需要同时处理来自多个树莓派客户端的请求。常见并发模型有三种：

- (1) 单线程轮询 (select/poll)，适合 IO 密集型场景，但需要手动维护状态机；
- (2) 线程池模型，主线程 accept 后将连接交给线程池中的工作线程，适合请求耗时较均匀的场景；
- (3) 一线程一连接模型，每个 TCP 连接分配一个独立线程，连接间天然隔离，编程模型简单。

本系统选择方案 (3)，主要基于以下考虑：

- 课程项目阶段客户端数量有限 (< 10 个);
- 每个连接的识别请求耗时主要由远端模型推理决定;
- 线程间隔离可以避免某个慢请求阻塞其他客户端。

(2) **多线程 TCP 接收循环**。主线程在端口上执行 `accept()` 阻塞等待新连接。每当新客户端接入, `TcpBridgeNode` 创建一个守护线程运行 `_client_loop()`, 该线程内循环接收消息、处理请求并返回响应。 `MultiThreadedExecutor` 确保 ROS2 回调在多线程环境下安全执行:

Listing 5: 服务端多线程 TCP 接收模型

```

1 class TcpBridgeNode(Node):
2     def __init__(self):
3         # ... ROS2 发布/订阅初始化 ...
4         self._server_socket = socket.socket(socket.AF_INET, socket.
5             SOCK_STREAM)
6         self._server_socket.setsockopt(socket.SOL_SOCKET, socket.
7             SO_REUSEADDR, 1)
8         self._server_socket.bind((self.host, self.port))
9         self._server_socket.listen()
10        self._accept_thread = threading.Thread(
11            target=self._accept_loop, name="tcp-bridge-accept", daemon=
12                True
13        )
14        self._accept_thread.start()
15
16    def _accept_loop(self):
17        while not self._stop_event.is_set():
18            client_socket, address = self._server_socket.accept()
19            thread = threading.Thread(
20                target=self._client_loop,
21                args=(client_socket, address),
22                daemon=True,
23            )
24            self._client_sockets.add(client_socket)
25            self._client_threads.add(thread)
26            thread.start()
27
28    def _client_loop(self, client_socket, address):
29        while not self._stop_event.is_set():
30            request, attachment = receive_message(client_socket)

```

```

28         response = self._handle_request(request, request_id,
29                                           attachment)
        send_json_message(client_socket, response)

```

(3) **连接管理与健康监控**。为防止资源耗尽，系统实现了三个关键机制：

- (1) **连接上限控制**。新增 `max_connections` 参数（默认 10），`accept` 时检查当前连接数，超出则拒绝新连接并关闭 `socket`；
- (2) **每连接统计跟踪**。使用 `ClientSession` 数据类记录每个连接的建立时间、请求次数、收发字节量和最后活跃时间；
- (3) **定时健康报告**。每 30 秒输出一次连接总览日志，便于运维排查。

Listing 6: 连接管理与会话跟踪

```

1  @dataclass
2  class ClientSession:
3      address: tuple[str, int]
4      connected_at: float
5      request_count: int = 0
6      error_count: int = 0
7      bytes_received: int = 0
8      bytes_sent: int = 0
9      last_active: float = field(default_factory=time.monotonic)
10
11  def _accept_loop(self):
12      while not self._stop_event.is_set():
13          client_socket, address = self._server_socket.accept()
14          if self.max_connections > 0:
15              with self._session_lock:
16                  active = len(self._sessions)
17                  if active >= self.max_connections:
18                      self.get_logger().warn(
19                          f"Rejected {address} — {active}/{self.
20                              max_connections} full"
21                      )
22                      client_socket.close()
23                      continue
24          # ... 创建 ClientSession, 启动 _client_loop 线程 ...

```

```

25 def _log_health_report(self):
26     """每隔 30 秒输出连接总览"""
27     with self._session_lock:
28         sessions = list(self._sessions.items())
29         for peer, session in sessions:
30             conn_sec = time.monotonic() - session.connected_at
31             logger.info(
32                 f" {peer} id={conn_sec:.0f}s req={session.request_count}
33                 "
34                 f"err={session.error_count} rx={session.bytes_received}B"
35             )

```

- (4) **优雅关闭**。节点销毁时按以下顺序清理资源：设置停止标志 → 关闭监听 socket → 依次 shutdown 所有客户端 socket → 等待 accept 线程退出 → 等待所有 client_loop 线程退出 → 清理待处理请求并返回 SHUTDOWN 错误码。这种顺序保证已建立连接能正常收到关闭通知，而非被粗暴断开。

3.5 ROS2 节点化识别

- (1) **节点模型原理**。ROS2 的节点模型将功能拆为独立进程或线程，节点之间通过 topic（发布/订阅）、service（请求/响应）或 action（带反馈的长时间任务）交换消息。发布者和订阅者只依赖消息类型，不直接依赖彼此实现，天然支持模块替换。
- (2) **服务端节点划分**。服务端保留 ROS2 Humble 包结构，核心 launch 文件 tcp_bridge.launch.py 同时启动四个节点：

- 物资识别节点；
- 人脸识别节点；
- TCP 网桥；
- Web 看板。

TCP 网桥将外部请求转为 ROS2 图像消息，识别节点订阅图像并发布 JSON 结果。拆分的主要收益有两点：

- 服务端硬件资源比客户端宽裕，适合集中安装模型库和人脸识别依赖；
- 节点化降低单个文件维护压力。旧版 integrated_server.py 725 行，当前最大服务端节点 tcp_bridge_node.py 约 410 行，人脸节点 face_recognize_node.py 99 行。

- (3) **请求关联机制**。网桥与识别节点之间通过 `request_id` 关联异步响应：网桥将 `request_id` 写入 `Image.header.frame_id`，识别节点返回结果时保留同一 `frame_id`，网桥据此在等待表（`_pending` 字典）中找到对应的 TCP 请求并返回结果。超时保护由 `threading.Event.wait(timeout)` 实现，默认 15 秒。

Listing 7: 网桥请求等待与结果回调

```
1 def _submit_image_request(self, publisher, request_id, image,
    response_type):
2     pending = PendingResponse(event=threading.Event())
3     self._pending[request_id] = pending
4     msg = bgr_to_image_msg(image, frame_id=request_id)
5     publisher.publish(msg)
6     if not pending.event.wait(self.request_timeout_sec):
7         raise ProtocolError("TIMEOUT", f"Timed out waiting for {
            response_type}")
8     return pending.response
9
10 def _complete_pending_from_result(self, payload_text, response_type):
11     payload = json.loads(payload_text)
12     request_id = payload.get("frame_id")
13     pending = self._pending.get(request_id)
14     if pending is not None:
15         pending.response = make_success_response(response_type,
            request_id, payload)
16     pending.event.set()
```

3.6 物资检测

- (1) **YOLO 单阶段检测原理**。YOLO (You Only Look Once) 将目标检测建模为回归问题：将输入图像划分为 $S \times S$ 的网格，每个网格预测 B 个边界框及对应的置信度 C 和类别概率 $P(\text{class})$ 。最终输出直接来自一次前向推理，没有候选区域生成 (RPN) 阶段。模型输出经过非极大值抑制 (NMS) 去除冗余框后，得到最终检测结果。
- (2) **计数转换**。物资识别节点在 `server/server/material_counter_node.py` 中实现，订阅 `/material_counter/image` 话题，在 `image_callback()` 中调用 `model.predict()` 进行推理。检测结果通过 `summarize_counts()` 按类别累加检测框数量，返回类似 `{"cboard": 2, "m3508": 1}` 的计数字典。节点默认通过 `resolve_default_model_path()` 查找权重，优先使用 `last.pt`，找不到时回退到 `best.pt`。相比早期方案中的二维码逐件扫描，YOLO

差分方式只需拍前后两张图，不需要为每件物资制作和维护标签。

Listing 8: 物资检测节点回调逻辑 (material_counter_node.py)

```
1 def image_callback(self, msg):
2     image = image_msg_to_bgr(msg)
3     predictions = self.model.predict(
4         source=image, conf=self.conf_threshold,
5         device=self.device, verbose=False,
6     )
7     result = predictions[0]
8     counts = summarize_counts(result)
9     payload = {
10         "frame_id": msg.header.frame_id,
11         "counts": counts,
12         "total_detections": int(sum(counts.values())),
13     }
14     self.counts_publisher.publish(
15         String(data=json.dumps(payload, ensure_ascii=False)))
```

3.7 人脸识别门控

- (1) **人脸编码比对原理。** face_recognition 库基于 dlib 的深度学习模型，将人脸图像映射为 128 维特征向量 (face encoding)。同一人的不同照片在编码空间中距离较近 (欧氏距离通常 < 0.6)，不同人的编码距离明显更大。识别过程可分为三步：

- (1) 用 HOG (Histogram of Oriented Gradients) 或 CNN 检测人脸位置；
- (2) 将检测到的人脸区域输入编码网络，得到 128 维向量；
- (3) 计算待识别人脸编码与库中已知编码的欧氏距离，取最小距离者，若低于阈值则匹配。

已知人脸库由 server/server/face_database.py 中的 load_known_face_database() 从磁盘加载并预编码。

- (2) **门控规则。** FaceRecognizeNode 在 server/server/face_recognize_node.py 中实现，订阅 /face_recognize/image 话题，在 image_callback() 中调用 recognize_faces() 进行编码比对，返回识别结果、计数字典和总人脸数。客户端只有在服务端返回 total_faces=1 且唯一姓名不是 unknown 时才继续。这条规则可以过滤以下几种不可靠场景：

- 画面中无人，可能是摄像头未对准；
- 画面中有多人，无法确定操作者；

- 画面中的人不在已知库中，说明当前无权限。

后续 CSV 中的每条记录都带有操作者姓名，追溯时不需要再反推“这次是谁登记的”。

Listing 9: 人脸识别节点回调逻辑 (face_recognize_node.py)

```

1 def image_callback(self, msg):
2     image = image_msg_to_bgr(msg)
3     results = recognize_faces(
4         image, self.known_encodings, self.known_names,
5         tolerance=self.tolerance,
6         detection_model=self.detection_model,
7         unknown_label=self.unknown_label,
8     )
9     counts = dict(sorted(Counter(
10         str(item["name"]) for item in results).items()))
11     payload = {
12         "frame_id": msg.header.frame_id,
13         "results": results,
14         "counts": counts,
15         "total_faces": len(results),
16     }
17     self.result_publisher.publish(
18         String(data=json.dumps(payload, ensure_ascii=False)))

```

3.8 库存记录与看板

(1) **记录原理。**库存记录将一次差分结果拆为多条追加式流水，每行包含以下字段：

- request_id;
- record_time;
- person;
- action (“入库” 或 “出库”);
- material_name;
- quantity;
- received_at。

TcpBridgeNode 的 `_write_inventory_record()` 方法负责将校验后的库存流水追加到 CSV 文件，使用 `_csv_lock` 线程锁保护文件写入，防止多客户端并发写入导致数据交错：

Listing 10: CSV 库存记录写入 (tcp_bridge_node.py)

```

1 def _write_inventory_record(self, request_id, record):
2     rows = [{
3         "request_id": request_id,
4         "record_time": record.record_time,
5         "person": record.person,
6         "action": record.action,
7         "material_name": item.name,
8         "quantity": item.quantity,
9         "received_at": datetime.now().astimezone().isoformat(),
10    } for item in record.items]
11    self.inventory_csv_path.parent.mkdir(parents=True, exist_ok=True)
12    with self._csv_lock:
13        write_header = (not self.inventory_csv_path.exists()
14                        or self.inventory_csv_path.stat().st_size == 0)
15        with self.inventory_csv_path.open("a", newline="", encoding="
16            utf-8") as f:
17            writer = csv.DictWriter(f, fieldnames=CSV_FIELDNAMES)
18            if write_header:
19                writer.writeheader()
20            writer.writerows(rows)
21    return {"csv_path": str(self.inventory_csv_path), "rows_written":
22        len(rows)}

```

- (2) **看板原理**。Web 看板基于 Python 标准库 `http.server` 和 `ThreadingHTTPServer`, 默认监听 `127.0.0.1:8600`。DashboardStore 定时读取 CSV, 通过 `server/server/inventory_dashboard.py` 中的 `build_dashboard_data()` 逐行解析流水记录, 按物资名称聚合当前库存、累计入库/出库量、交易次数和最后操作者, 并根据 `get_stock_status()` 判定库存状态 (待补货/偏低/平稳/充足)。前端收到 `/api/summary` 后渲染库存总览表和最近动态, 并通过 SSE (Server-Sent Events) 推送增量更新。看板让演示时不必打开代码或 CSV 就能确认系统是否完成了记录闭环。

四、数据集与模型训练

4.1 数据准备

数据集采用 YOLO 检测格式, 类别由 `configs/classes.yaml` 定义: cboard (控制板)、dmj4310 (电机)、m3508 (减速电机)。仓库中实际图像 61 张, 划分为训练集 36 张、验证集 18

张、测试集 7 张，如表 3 所示。数据采集覆盖了不同距离、角度、光照和背景条件，标注使用 labelImg 工具，框紧贴物体外接边界，遮挡但仍可辨认的目标继续标注。

表 3: 数据集划分

划分	图像数	用途	类别数
训练集	36	更新模型参数	3
验证集	18	观察训练过程	3
测试集	7	检查最终效果	3

4.2 训练过程与结果

训练脚本 scripts/train.py 以 yolo11n.pt 为预训练权重启动微调,默认训练 100 个 epoch,每 10 个 epoch 保存一次权重,支持断点续训。模型导出脚本已支持 ONNX 和 TorchScript 格式。

训练指标随 epoch 变化如图 4 所示。模型在前半段收敛较快,第 31 轮取得了本文报告的综合最优指标: Precision 80.21%, Recall 80.36%, mAP50 87.74%, mAP50-95 76.15% (见表 4)。后续训练存在一定波动,这与当前样本量偏小有关。

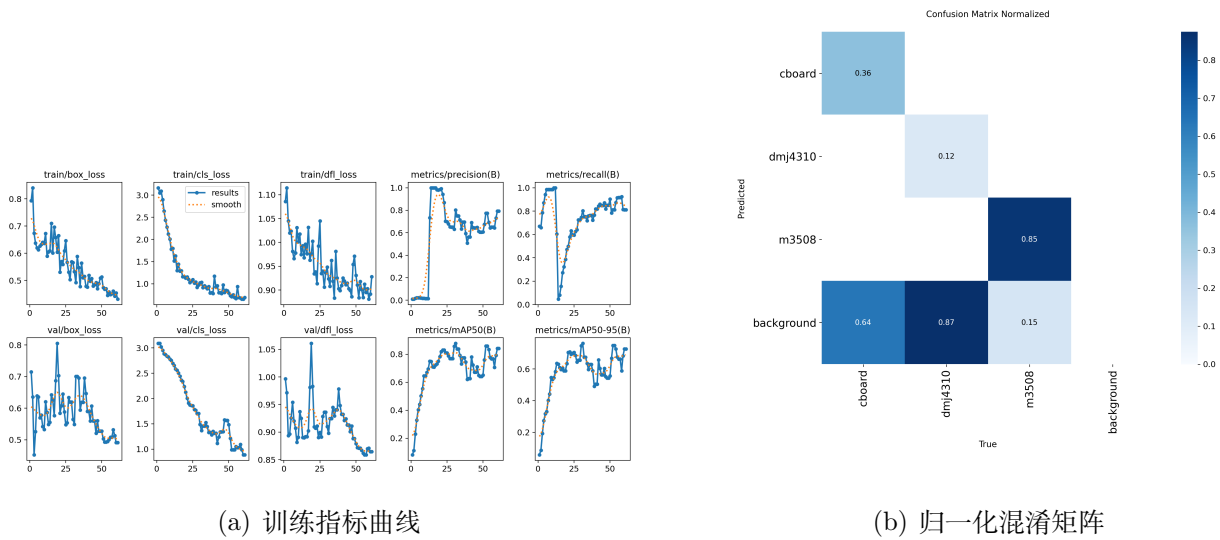


图 4: 训练结果

表 4: 第 31 轮模型指标

Epoch	Precision	Recall	mAP50	mAP50-95
31	0.8021	0.8036	0.8774	0.7615

图 5 展示了单图检测效果，该图对应的计数输出为 cboard: 2、m3508: 1。在系统流程中，这类计数输出被保存为前态或后态计数字典并参与库存差分。

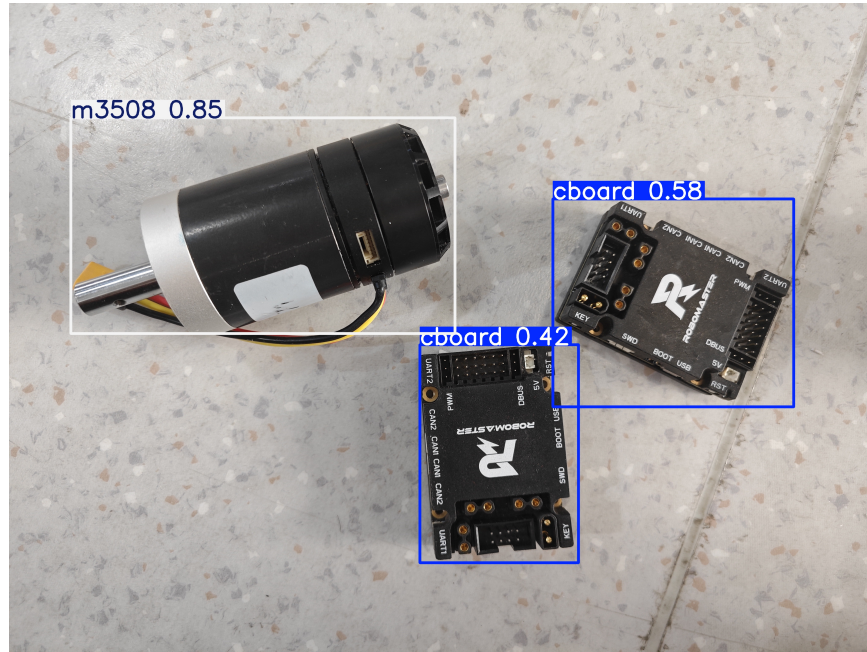


图 5: 单图检测结果示例

五、项目迭代历程

5.1 版本演进

项目并非一开始就采用当前架构，而是经历了几轮调整。表 5 按时间顺序概括了主要迭代阶段。

表 5: 版本演进

阶段	代表实现	内容与收益
基础验证	人脸、TCP、二维码识别	先完成单点能力验证，确认硬件、网络和识别库能在目标环境运行。
单机闭环	SystemUI	将人脸识别、二维码扫描、舵机开门和 CSV 记录集成在一个 Tkinter 程序中，适合快速演示但耦合较重。
端—服联调	旧客户端 + 旧服务端	拆为客户端和服务端，协议支持 RECOGNIZE、DETECT_MATERIALS、DATA_JSON，形成”认证—前态—后态—提交”雏形。
YOLO 替代 QR	YOLO 独立目录	新增 YOLO 训练/验证/导出/检测流程，从二维码逐件扫描转向前后图像差分计数。
当前重构	client/ + server/server/	服务端改造为 ROS2 节点组合，客户端改为不依赖 ROS2 的 Python 包，协议统一为长度前缀 JSON + 二进制附件，新增多客户端并发支持和健康监控。

5.2 关键取舍

表 6: 关键迭代的抉择与依据

抉择	原因	依据
QR → YOLO 差分	二维码方案对每个物资都需要贴码和维护，yolo 识别在训练后对每一类的新增物资无需特殊管理	3 类 61 张图片，31 轮 mAP50 87.74%；用户只需拍前后两张图
单体服务端 → ROS2 节点	单体脚本功能越多越难拆开调试	旧版 725 行降至当前最大节点 410 行，各节点彼此独立，也可通过 launch 组合启动
端侧 ROS2 → TCP 客户端	树莓派内存和存储较小，需要保持轻量化	客户端依赖仅 2 个，协议暴露 3 类请求，服务端内部仍用 ROS2
单连接 → 多线程并发	多个物资管理客户端需同时接入	新增连接上限参数、会话跟踪和健康报告，支持 10 路并发
自动前态 → 手动抓拍	系统目前只有一个摄像头，摄像头移动、遮挡和光照影响稳定性判断	联调经验，手动确认减少了”摄像头未对准就抓拍”的风险
只写 CSV → CSV + 看板	CSV 便于调试，但展示库存不直观	看板从同一份 CSV 聚合数据，便于定期检查

六、系统评估

6.1 系统特点

从课程项目的完成度来看，当前系统有以下几个比较明确的特点：

- (1) **服务端内部结构清晰**。服务端继续使用 ROS2 来组织功能模块，人脸识别、物资计数、TCP 网桥和库存看板互相解耦。相比早期“大脚本包办一切”的写法，现在更容易单独调试某个环节，也更方便后续替换模型或新增接口。
- (2) **人脸门控策略简单但实用**。本系统没有追求复杂的考勤式人脸系统，而是抓住“出入库必须绑定责任人”这一核心目标，采用“恰好一张已知人脸才允许继续”的门控规则。对实验室场景来说，这个策略实现成本低、行为边界清楚，足以满足谁在操作物资的追溯需求。
- (3) **客户端部署门槛较低**。客户端脱离 ROS2 后，端侧只需普通 Python 依赖即可运行，并能在树莓派上优先调用 `picam-still`。再结合上一节的桌面自启动设计，使用者不需要理解 ROS2 工作区、DDS 或复杂命令行，部署难度明显下降。

- (4) **具备基础的多客户端并发能力。**服务端 TCP 网桥采用一线程一连接模型，主线程 `accept` 后将连接交给独立守护线程，各连接之间互不干扰。配合连接上限、会话统计和定时健康报告，系统已经具备基本的分布式部署能力，多个实验室门禁点的树莓派可以同时接入同一服务端。
- (5) **跨设备通信更贴合现场布置。**客户端与服务端之间只依赖 TCP，不要求跨机器 ROS2 通信，也不要求设备必须放在同一张桌子上。对于实验室这种“树莓派靠近物资柜，推理主机放在别处”的场景，网络拓扑更灵活，传输优化后的带宽占用也更容易接受。

6.2 不足与改进方向

项目大体可以运行，但仍有改进空间：

- (1) 物资检测目前只覆盖三类目标，数据集 61 张偏小。后续应扩充样本，补充强反光、遮挡、相似背景和密集摆放场景；
- (2) 人脸库成员较少，光照和姿态变化需要更充分测试；
- (3) CSV 存储不适合长期使用，多人长期使用时应迁移到 SQL 类数据库，并补充权限控制、异常审核和备份机制；
- (4) 当前只有一个摄像头，权衡之下物资拍摄仍需手动触发；后续可使用双摄方案，一个用于人脸拍摄，一个用于物资拍摄；
- (5) 当前一线程一连接模型在客户端数量 > 50 时线程开销较大，未来可考虑改用 `asyncio` 或线程池模型。

七、总结

本项目完成了一套从图像采集到库存展示的分布式智能物资管理系统。系统的价值体现在将 YOLO 识别、人脸门控、TCP 通信、多线程并发、库存差分、CSV 落盘和 Web 看板串联为一个完整闭环。当前实现证明了“轻量客户端 + 集中式服务端 + 本地可视化看板”的路线在课程项目阶段具有可行性。系统的模块化结构和清晰的接口边界为后续扩展更多物资类别、数据库存储和更自动化的端侧交互预留了空间。

项目仓库结构与代码接口见表 7。

表 7: 主流程与源码对应关系

流程环节	关键实现	作用
UI 交互	<code>ui_node.py</code> / <code>ClientWindow</code>	三个按钮分别调用开始、物资抓拍和完成，UI 不直接处理识别逻辑
状态控制	<code>logic_node.py</code> / <code>LogicNode</code>	用状态和 token 控制异步请求，避免旧请求污染新一轮流程
差分计算	<code>logic_utils.py</code>	对前后计数字典逐项求差，生成入库/出库条目
请求构造	<code>tcp_client_node.py</code>	限定三类请求类型，为每次请求生成 <code>request_id</code>
TCP 编解码	<code>tcp_protocol.py</code> (两端共用)	双长度前缀切分 JSON 头和二进制图像体
TCP 并发管理	<code>tcp_bridge_node.py</code> / <code>accept</code> 循环 + <code>client_loop</code>	一线程一连接模型，带连接上限、会话跟踪和健康报告
服务端分发	<code>tcp_bridge_node.py</code> / <code>_handle_request()</code>	按 <code>type</code> 分发到识别节点或库存写入
识别节点	<code>face_recognize_node.py</code> 与物 资节点	订阅图像 topic，发布识别结果 JSON
记录与看板	<code>tcp_bridge_node</code> + <code>inventory_web_node</code>	前者追加 CSV，后者读取并聚合展示

参考资料

- [1] Ultralytics Docs. <https://docs.ultralytics.com/>
- [2] ROS 2 Humble Documentation. <https://docs.ros.org/en/humble/>
- [3] face_recognition Documentation. <https://face-recognition.readthedocs.io/en/latest/>
- [4] 本项目代码仓库: README.md、docs/project_plan.md、docs/annotation_guide.md 等工程文档。